PRÉ-PAPER · RASCUNHO

Programação Agêntica no Espaço de Pesos como Alternativa ao Fine-Tuning em Modelos de Linguagem Pequenos

Autores: Luiz Felipe Melo · **Afiliação:** Universidade do Sul de Santa Catarina (Unisul) · **Rascunho:** 13 nov. 2026 · **DOI:** 10.5281/zenodo.24208535

TL;DR

Modelos pequenos que "pensam" antes de responder costumam ficar presos repetindo a mesma coisa: *"Espera, deixa eu conferir… Sim… Espera…"*. Demos a agentes Claude os pesos do Qwen3.5-0.8B e um pequeno orçamento de testes, e pedimos que resolvessem isso editando os pesos diretamente, sem treinamento. **Com no máximo quatro edições de posto 1, o modelo passou de 15,5% para 61,5–68% em problemas trancados de matemática do ensino fundamental.** Isso iguala a edição que o GPT-6 "Astra", um modelo de fronteira, escreveu no estudo *Sparks of Weight-Space Programming*, do MIT CSAIL (Takehi et al., 2026), com cerca de metade dos tokens, e supera uma receita barata de fine-tuning. Mas alternativas simples chegam perto: desligar o raciocínio resolve 58% a uma fração do custo. O método funcionou num defeito concentrado e falhou num espalhado.

Tudo rodou em uma única placa de vídeo de jogos (GTX 1660 SUPER, 6 GB).

Um agente consegue consertar um modelo editando seus pesos. Em cerca de duas horas por execução, agentes Claude leram os pesos "através do vocabulário", acharam neurônios que empurram o modelo a dizer "Wait" e mudaram a força com que eles escrevem. Seis execuções do mesmo protocolo ficaram todas entre 58% e 67% nos problemas de prática, onde o modelo original resolve 15%.

As edições dos agentes precisam de menos tokens. A edição do Astra no *Sparks* (Takehi et al., 2026) chega ao mesmo acerto quando tem 20 mil tokens para pensar; as edições dos agentes já estão quase lá com 8 mil. Desligar o raciocínio sai ainda mais barato, e nos 400 problemas trancados as edições ficam só 6,5–9 pontos acima dele.

Testes reais importaram mais que ferramentas engenhosas. Toda nota rápida que demos aos agentes acabou sendo burlada, e só os testes reais os mantiveram honestos. Os agentes que receberam mais dados de interpretabilidade acharam correções piores nos três pares que rodamos.

Um modelo pequeno que entra em loop costuma ser tratado como um problema de treinamento: juntar exemplos de bom raciocínio e fazer fine-tuning. Para um modelo em loop, até juntar os exemplos é caro, porque a maioria das tentativas dele nunca termina.

Em *Sparks of Weight-Space Programming*, Takehi et al. (2026), do MIT CSAIL, mostraram outro caminho. O modelo de fronteira que eles usaram, o GPT-6 "Astra", leu os pesos brutos deste mesmo modelo Qwen, editou 0,016% deles, e o acerto no MATH-500 foi de 7,9% para 48,1%. Neste paper, "edição do Astra" ou "edição publicada" sempre quer dizer essa edição, que recuperamos com exatidão (Experimento 1). Fizemos três perguntas. Isso se reproduz numa montagem barata, com um agente Claude como editor? É mais que um truque que um ajuste de decodificação ou um fine-tuning comum igualaria? E o que, no processo do agente, realmente importa?



Figura 1: Um problema de prática real, mesma semente aleatória. O modelo original chega à resposta cedo e depois repete "Wait, 120 / 8 = 15" até ficar sem espaço. A versão de 2 edições da correção do segundo agente, a que publicamos, confere uma vez, fecha o raciocínio e responde. Linha de baixo: o modelo original no primeiro teste trancado; o modelo editado e o modelo sem edição com o raciocínio desligado no segundo.

O defeito: um modelo que não consegue parar de pensar

O Qwen3.5-0.8B "pensa" num rascunho antes de responder e o fecha com um marcador especial, `</think>`. Com um orçamento de 12.288 tokens (cerca de 9.000 palavras), a maioria das respostas nunca chega lá.

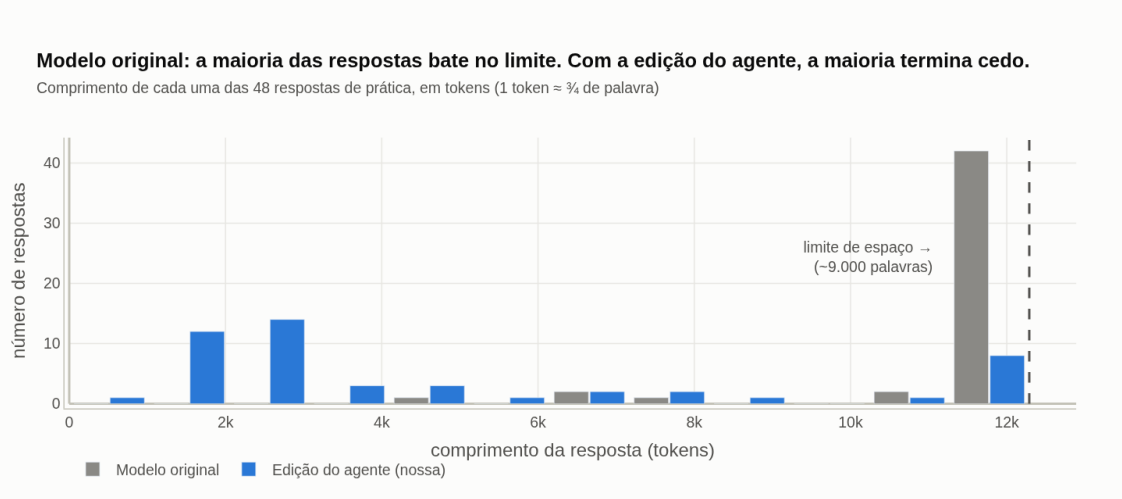


Figura 2: Comprimento de cada uma das 48 respostas de prática. A maioria das respostas do modelo original bate no limite de espaço; com a edição do agente, a maioria termina cedo.

Prompts não ajudam. Um system prompt mandando o modelo não se repetir não faz melhor que um prompt placebo sem sentido; os dois até *aumentam* um pouco a taxa de loop. Seja o que for que mantém o modelo em loop, as instruções não chegam até lá.

As regras do jogo

Configuração

- **Modelo e tarefa:** Qwen3.5-0.8B (Qwen Team, 2026) rodando no llama.cpp (Gerganov et al., 2023), resolvendo matemática do ensino fundamental (GSM8K; Cobbe et al., 2021) com raciocínio ligado e orçamento de 12.288 tokens.
- **Edições:** no máximo 4. Cada uma escolhe um grupo de neurônios (colunas da matriz de saída de uma camada), uma direção u no espaço interno do modelo e uma intensidade α, e muda só a força com que esses neurônios escrevem ao longo de u: $w_j \leftarrow w_j - \alpha\,(u^\top w_j)\,u$, com $|\alpha| \leq 10$. $\alpha = 1$ apaga essa parte, $\alpha > 1$ a inverte, $\alpha < 0$ a reforça.
- **O que os agentes recebem:** os pesos, um briefing que descreve os loops e um avaliador com 40 "notas rápidas" (o quanto o modelo *quer* se repetir, medido em segundos) e 6 "testes reais" (o modelo editado resolve de fato 16 problemas de prática, cerca de 10 minutos cada).
- **Não é permitido:** treinar, rodar o modelo fora do avaliador, editar as camadas de vocabulário, ou ler a edição do estudo original, as nossas anotações ou o trabalho de outros agentes. Auditamos depois o registro de cada agente e não achamos violações.
- **Agentes:** Claude Opus 5.5, cerca de 2 horas por execução. Algumas execuções também receberam um *atlas*, um mapa detalhado da atividade interna do modelo durante os loops.
- **Como julgamos:** cada escolha é confirmada em outros 48 problemas de prática; as comparações finais usam dois testes trancados de 200 problemas do GSM8K cada, abertos uma única vez depois de anotar as previsões, e 148 problemas de competição (MATH-500; Hendrycks et al., 2021; Lightman et al., 2024) que nenhuma busca tocou. Todos os métodos respondem aos mesmos problemas com as mesmas sementes, então dá para contar, problema por problema, quem resolveu o quê.

Um agente consegue corrigir os loops editando pesos?

EXPERIMENTO 1

Conseguimos reproduzir a edição do Astra?

Sim, com exatidão. A edição publicada por Takehi et al. (2026) muda duas matrizes, nas camadas 15 e 19. Em cada uma, pega o que 64 neurônios escrevem ao longo de uma "direção de repetição" e inverte, deixando em $-5\times$ o tamanho original. Na nossa montagem, ela leva o conjunto de prática de 15% para 46% e a matemática mais difícil de 8% para 21%. Nossos números absolutos no MATH são menores que os do estudo original porque o nosso orçamento é de 12 mil tokens, e não 32 mil, e isso pesa mais para essa edição: ela ainda está melhorando quando bate nos 12 mil.

EXPERIMENTO 2

Um agente Claude consegue encontrar a própria correção?

Dois agentes, trabalhando de forma independente, chegaram à mesma receita em duas partes: deixar "Wait" menos provável e </think> mais provável. Ela resolve 67% e 65% do conjunto de prática. Um primeiro agente, só com as notas rápidas, empatou com a edição publicada (44% contra 41% em 32 problemas de teste). Acrescentar os 6 testes reais fez a diferença.

agente	o que a edição faz ($\leq$ 4 edições de posto 1)	conjunto de prática
a1	reforça o freio sobre "Wait" em 3 neurônios (L17n1915, L18n881, L22n573) e o sinal de </think> na última camada	**67%** (26 resgatados, 1 perdido)
a2	inverte o que 4 neurônios escrevem na direção de "Wait" (L16n998, L16n3422, L17n1915, L22n573) e reforça </think> na camada 22	**65%** (24 resgatados, 0 perdidos)

Como fizeram: os dois agentes leram os pesos "através do vocabulário", perguntando que palavra cada neurônio empurra o modelo a dizer. As anotações mostram que aprenderam com os erros. Apagar de vez os neurônios de "Wait" fez o modelo dizer "wait wait wait…" e não resolver nada, o que lhes mostrou que esses neurônios funcionam ao contrário do que tinham suposto.

Num loop, as duas portas precisam mudar. Um loop precisa de um caminho de volta ("Espera, deixa eu conferir") e de nenhuma saída. Nos testes reais, o reforço de </think> sozinho resolve 19%, a supressão de "Wait" sozinha 44–50%, e as duas juntas 69%.

Mesmo circuito, alavancas diferentes. Em 12 textos em loop do modelo original, com o texto fixo:

- **A a1 reforça um freio.** O neurônio L22n573 dispara negativo antes de todo "Wait" (52 de 52) e escreve contra a palavra; a a1 deixa esse empurrão 7$\times$ mais forte.
- **A a2 tira o empurrão mais acima.** Quase todo o efeito dela sobre "Wait" vem de um neurônio, o L16n998, que promove "Wait" no modelo original e o suprime depois da edição. Sem esse empurrão, o freio da camada 22 se cala.
- **Por isso a a2 precisa de só 2 edições.** Mantendo só o L16n998 e a edição de </think>, o modelo resolve 69% do conjunto de prática e 67,5% do segundo teste trancado (a2 inteira: 64,0%). É a versão que publicamos. Só o L16n998 não basta: 25 de 48 respostas ainda ficam sem espaço.

(Qual variante da a1 relatamos, e por quê, está no Apêndice A.)

EXPERIMENTO 3

A correção se mantém em problemas que o agente nunca viu?

Na matemática de competição, que nunca foi usada em nenhuma busca, a edição resolve 47%, contra 21% da edição publicada no mesmo orçamento (44 problemas resgatados, 6 perdidos). A edição do segundo agente resolve 49%.

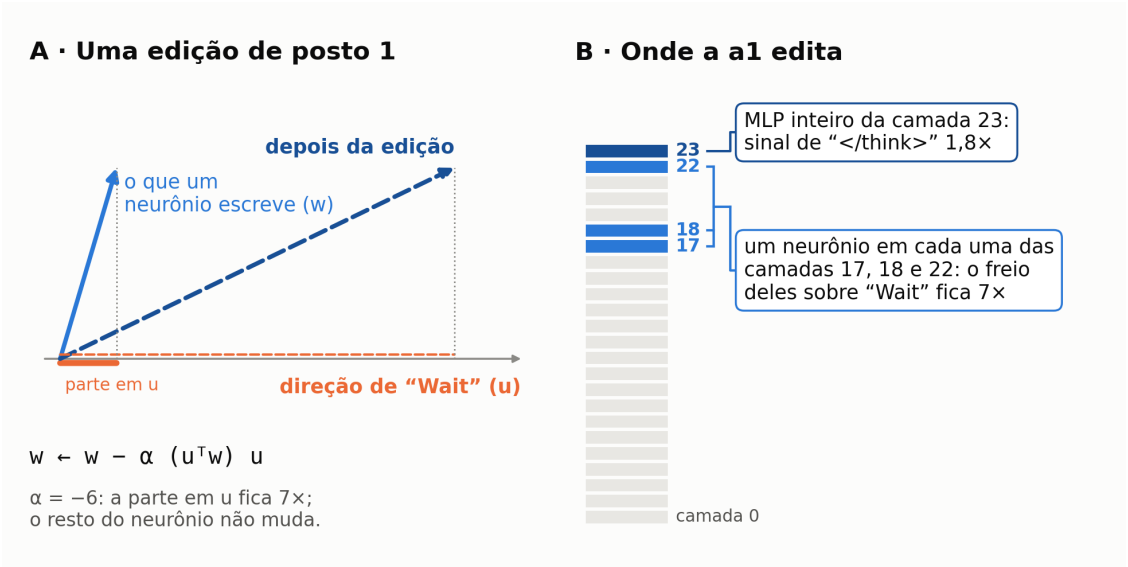


Figura 3: Anatomia da edição a1. (A) Uma edição de posto 1 muda só a parte do que um neurônio escreve numa direção escolhida. (B) A a1 reforça o freio de três neurônios sobre “Wait” e o sinal de `</think>` na última camada.

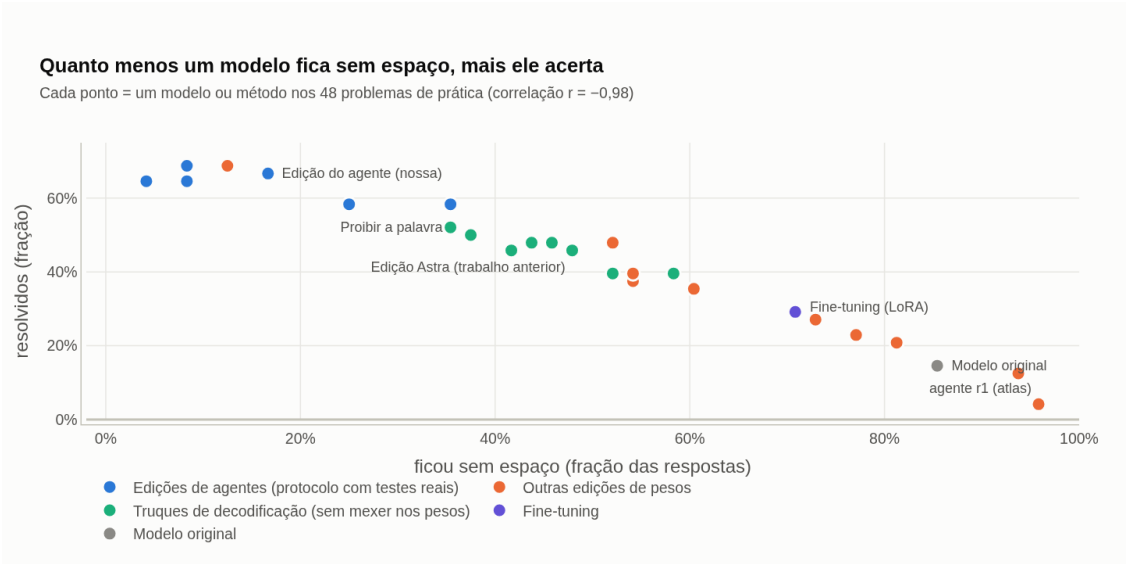


Figura 4: Em 27 modelos e métodos que rodamos no conjunto de prática, quanto menos respostas ficam sem espaço, mais respostas estão certas (r = −0,98).

Nos testes trancados, as edições resolvem 61,5–68%, contra 15,5% do modelo original. O primeiro teste trancado teve braços acrescentados depois da abertura, então abrimos um segundo com todos os braços e previsões fixados antes.

método	teste trancado 1	teste trancado 2	tokens por resposta
modelo original	15,5%	—	11.142
edição publicada (Astra)	53,5%	—	8.877
modelo original, raciocínio desligado	58,5%	58,0%	cerca de 1.300
edição do agente (a1)	61,5%	68,0%	cerca de 4.800
edição do segundo agente (a2)	67,5%	64,0%	cerca de 4.500
a2 reduzida a 2 edições (publicada)	—	67,5%	4.217

Cada teste tem 200 problemas, então um resultado isolado varia ±3–4 pontos. Somando os

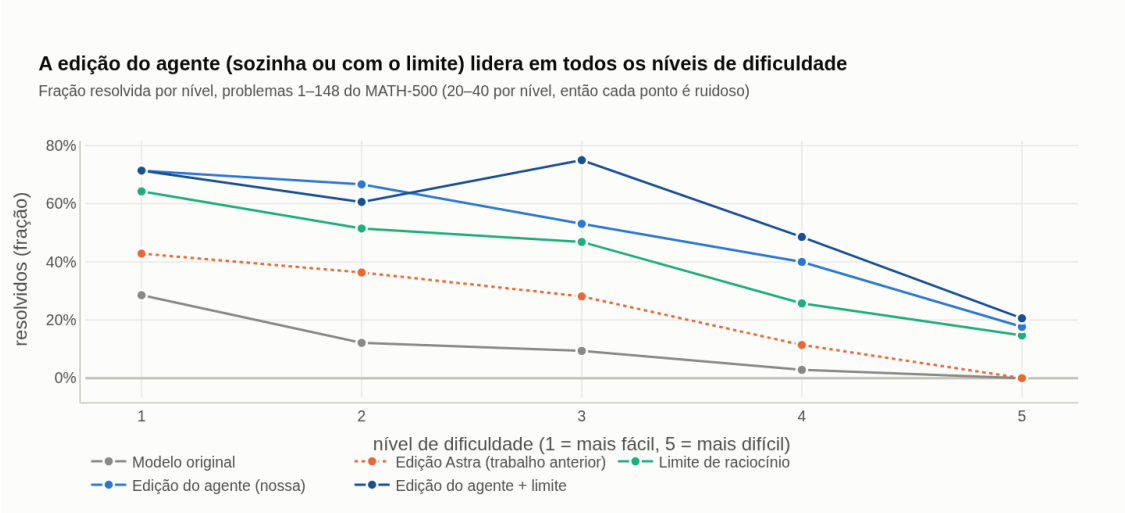


Figura 5: A edição do agente (sozinha ou com o limite de tempo de raciocínio) fica à frente em todos os níveis de dificuldade.

dois testes, as edições ficam 6,5–9 pontos acima do raciocínio desligado. O Apêndice A traz as comparações pareadas e os valores de p.

Com um orçamento maior, a edição publicada alcança

Com 12 mil tokens, a edição publicada ainda está subindo: 42,5% das respostas dela continuam em andamento no teto. Então rodamos de novo ela, a a1 e a a2 com teto de 20.480 tokens.

Com 20 mil, as três empatam: 65,0% para a edição publicada, 68,0% para a a1 e 63,5% para a a2. Mas as edições dos agentes chegam lá cedo. Quase não ganham nada depois de 12 mil, e com 8 mil já resolvem 61–63,5%, onde a edição publicada resolve 40%. No mesmo teto, usam 1,7–1,9× menos tokens.

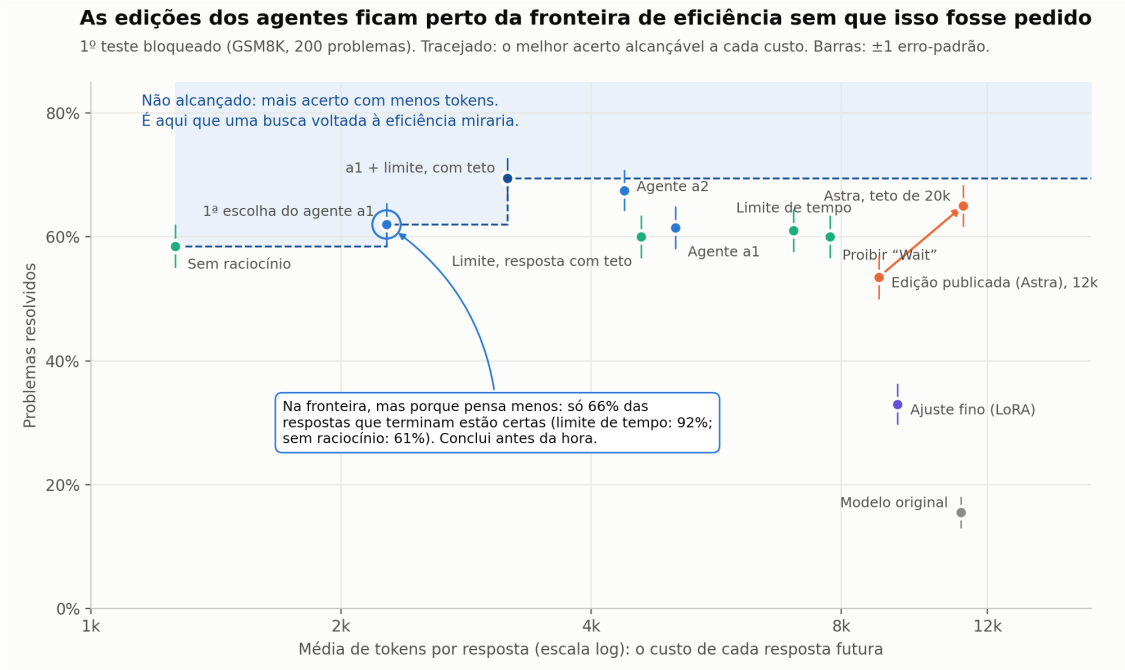


Figura 6: Acerto contra tokens por resposta no primeiro teste trancado. A linha tracejada liga os métodos que nenhum outro supera nos dois eixos. Nunca pedimos aos agentes que economizassem tokens, e mesmo assim as edições deles ficam perto dessa linha. A primeira escolha do agente a1 está na linha porque pensa menos, e mais respostas terminadas dela estão erradas.

Uma rodada não basta para ordenar duas boas edições. Rodar de novo a mesma edição, com as mesmas sementes, a moveu em até 5,5 pontos, porque o servidor agrupou os pedidos de outro jeito. A a2 ficou à frente da a1 numa rodada e atrás na seguinte, então não as ordenamos.

Quão repetível é?

Rodamos o protocolo vencedor mais quatro vezes, e toda execução consertou o defeito.

execução	editor	o que a escolha faz	conjunto de prática
a1	Opus	reforça 3 neurônios-freio de “Wait”, mais `</think>`	67%
a2	Opus	inverte o L16n998 (e outros 3 neurônios), mais `</think>`	65%
a3	Opus	inverte uma direção de “hesitação” (“Wait”, “Hmm”, “Actually”…) em quatro camadas inteiras	58%
a4	Opus	reforça 3 neurônios-freio de “Wait” (L18n881, L20n405, L22n573), mais `</think>`	58%
f1	Fable	edições de camada inteira na direção de “Wait” nas últimas camadas	65%
f2	Fable	edições de camada inteira na direção de “Wait” nas últimas camadas	60%

As seis ficam entre 58% e 67% (média de 62%), então a a1 e a a2 estão no topo da faixa. A a4 redescobriu sozinha o mecanismo da a1: primeiro inverteu os neurônios-freio, obteve “wait wait wait…” e depois passou a reforçá-los. As execuções do Fable (Claude Fable 5.2) foram exploratórias: pausadas, retomadas e com orçamento estendido. Mostram que outro editor consegue consertar o defeito por outro caminho; não dizem qual editor é melhor.

É só um truque?

Talvez a edição seja um jeito indireto de fazer algo mais simples. Testamos as coisas mais simples, cada uma ajustada em 48 problemas, três vezes o que os agentes viram.

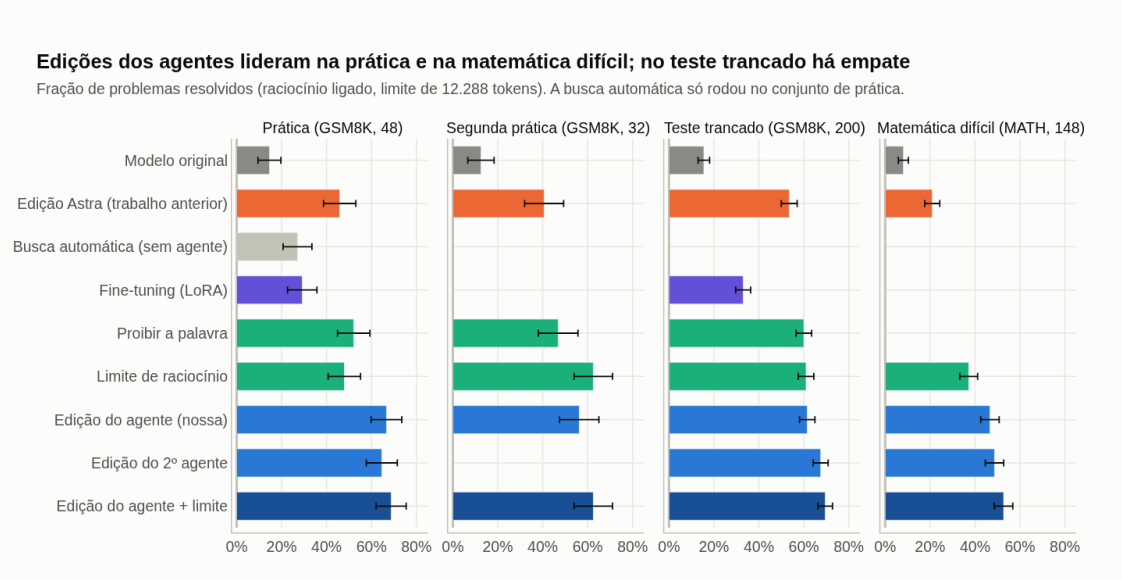


Figura 7: Parcela de problemas resolvidos por cada método nos quatro conjuntos de problemas (barras de erro: ±1 erro padrão). A tabela completa está no Apêndice A.

EXPERIMENTO 4

Um ajuste de decodificação faria o mesmo?

Configuração

- **Proibir a palavra:** enquanto o modelo escreve, tornar "Wait" e suas variantes quase impossíveis e dar um empurrão em `</think>`.
- **Limite de raciocínio** ("budget forcing"; Muennighoff et al., 2025): depois de 4.096 tokens de raciocínio, cortar e forçar uma resposta.
- **Raciocínio desligado:** deixar o modelo sem edição responder direto.

Na matemática do ensino fundamental, os truques simples chegam perto. No primeiro teste trancado, proibir a palavra (60,0%), o limite de raciocínio (60–61%) e o raciocínio desligado (58,5%) empatam com a a1 (61,5%). Somando os dois testes trancados, as edições abrem 5–9 pontos, mas o raciocínio desligado usa só um terço dos tokens delas.

Na matemática mais difícil, a edição se sai melhor que forçar uma resposta. Ela vence o limite de raciocínio por 9,5 pontos (23 resgatados, 9 perdidos). Forçar a resposta funciona quando a resposta já estava calculada antes do loop começar, o que é comum em problemas do ensino fundamental e mais raro nos mais longos. O raciocínio desligado (44%) empata com as edições sozinhas; só a edição combinada com o limite (53%) fica acima dele.

A edição e o limite se somam. A edição tira a maior parte dos loops e o limite pega o que sobra. Na matemática mais difícil, a combinação ganha 6 pontos sobre a edição sozinha. Na matemática do ensino fundamental, o ganho não ficou estabelecido.

EXPERIMENTO 5

Um fine-tuning comum faria o mesmo?

Configuração

- **Dados:** o raciocínio do próprio modelo em 600 *outros* problemas do GSM8K. Só 57 tentativas terminaram limpas e certas; em 279 que entraram em loop, cortamos o raciocínio onde o loop começava e deixamos o modelo responder, ficando com as 118 certas.
- **Treino:** LoRA (Hu et al., 2022), posto 16, todas as camadas, 3 rodadas, a melhor escolhida nos 16 problemas dos agentes.

O fine-tuning chegou a 29% do conjunto de prática, com 13× o tempo de GPU de uma execução de agente. Problema por problema, a edição do agente resgatou 18 problemas que ele errou e não perdeu nenhum. O modelo com fine-tuning ainda ficou sem espaço em 71% das respostas: aprendeu a responder bem depois que o raciocínio é cortado, mas nunca aprendeu a parar sozinho. É uma receita barata; mais dados ou aprendizado por reforço talvez façam melhor, a um custo maior.

EXPERIMENTO 6

O agente é necessário, ou bastam as ferramentas dele?

Um script com a caixa de ferramentas e o orçamento dos agentes, mas sem raciocínio, chegou a 27% do conjunto de prática, 40 pontos abaixo dos agentes. Ele testava edições aleatórias nas direções das 300 palavras mais comuns. Mas existem buscas sem LLM mais fortes: as nossas próprias edições por diferença de médias recuperam cerca de dois terços do ganho da edição publicada, e trabalhos concorrentes recuperam 79–93% (Zhang, 2026; nathanrs, 2026); um conserto às cegas, com um editor mais fraco e sem descrição do defeito, não achou nada (WKJBryan, 2026). O que dá para afirmar é mais estreito: os nossos agentes superaram todas as buscas sem agente que rodamos, e o briefing deles dizia qual era o defeito.

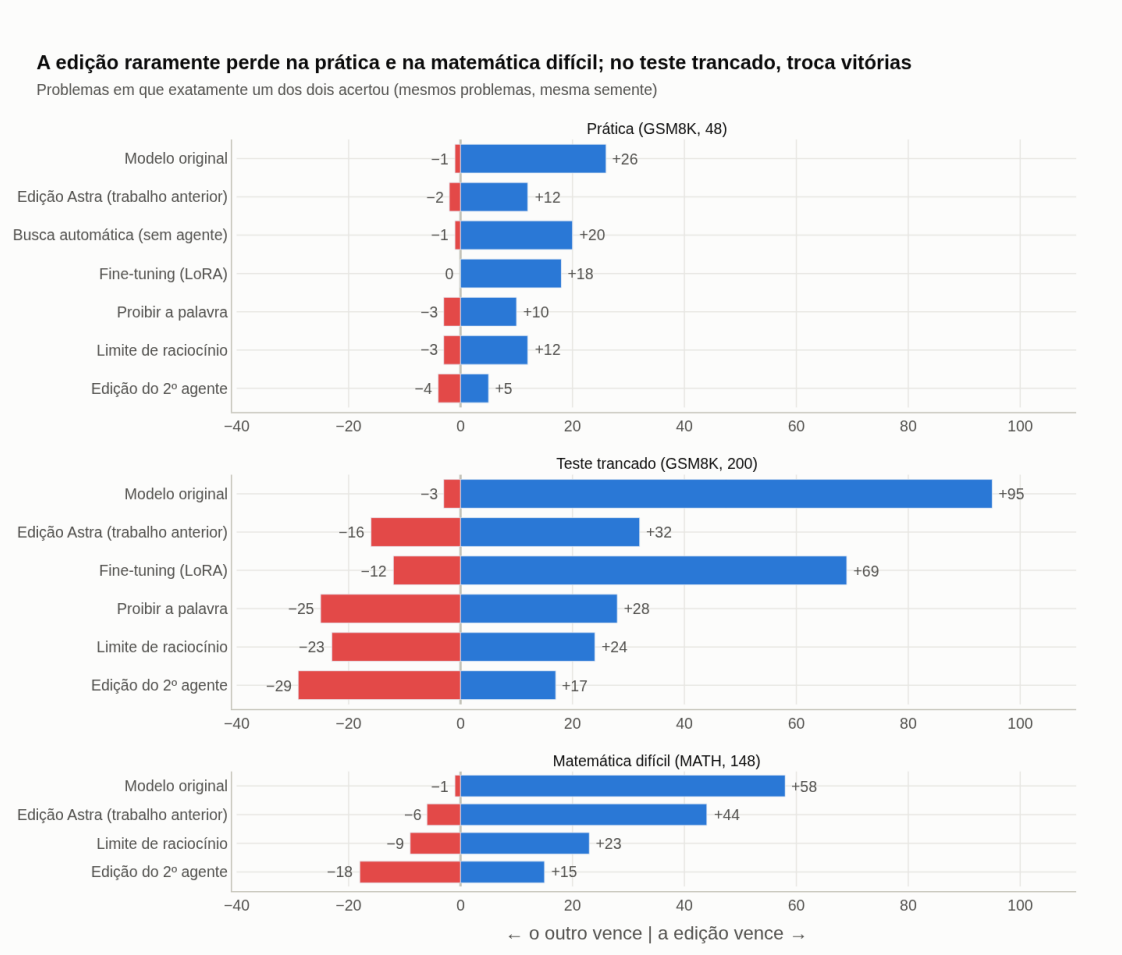


Figura 8: Problema por problema, mesmos problemas e sementes: problemas que só a edição do agente resolveu (à direita) contra problemas que só o outro método resolveu (à esquerda).



Figura 9: Os 6 testes reais de cada buscador (16 problemas cada). Os dois agentes sem atlas chegaram a 11 de 16; os agentes com atlas, a 5–7; o script, a 3.

O que aprendemos sobre o processo

EXPERIMENTO 7

Uma nota rápida pode substituir os testes reais?

Não. Toda nota rápida que testamos quebrou quando um otimizador forte a pressionou. O agente r1 levou a nota rápida a 20× o valor da edição publicada, e a edição dele resolveu 4% dos problemas, pior que não fazer nada: bloqueou a cópia palavra por palavra, e o modelo simplesmente entrou em loop de outros jeitos. Outra edição, sem nenhum loop palavra por palavra, ainda ficou sem espaço 77% das vezes. Seis testes reais resolveram isso; nesse protocolo, os dois agentes abandonaram ideias iniciais que tinham boa nota mas resolveram 0 de 16. **O objetivo de um agente precisa vir do comportamento real, mesmo de uma amostra pequena dele.**

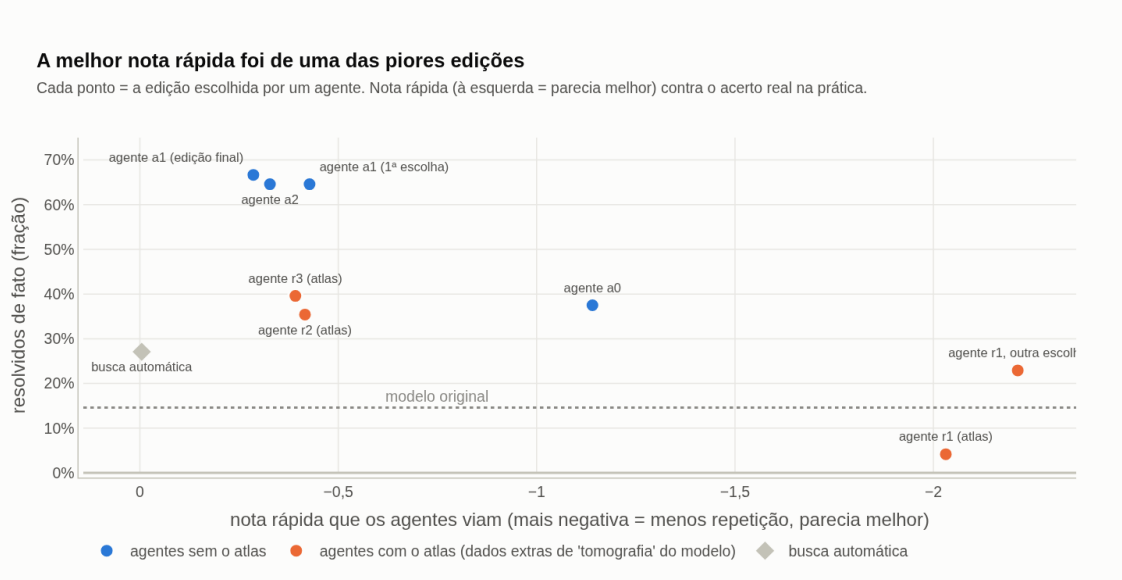


Figura 10: A nota rápida que a edição escolhida de cada agente alcançou (à esquerda = parecia melhor) contra a frequência com que ela de fato resolveu problemas.

EXPERIMENTO 8

Mais dados de interpretabilidade ajudam?

Nos três pares de agentes com instruções idênticas, o que não tinha o atlas achou a edição melhor (38% contra 4%, 67% contra 35%, 65% contra 40%). Com o atlas, os agentes perseguiram mecanismos intrincados, como um estado de "reconferência" nas camadas do meio, que pareciam convincentes mas não paravam os loops. Sem ele, foram direto às alavancas. Três pares são poucos para provar isso, e tínhamos previsto o contrário.

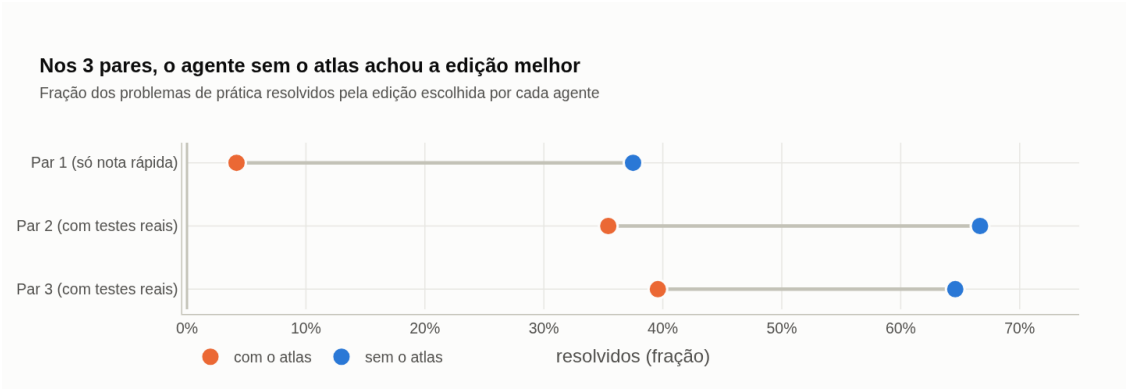


Figura 11: Cada par de agentes tinha instruções idênticas; o agente sem o atlas se saiu melhor todas as vezes.

Quanto custou?

	conjunto de prática	horas de GPU (1660 SUPER)	uso do agente
edição do agente (uma execução)	**67%**	**1,3**	5,4 M tokens lidos, 48 mil escritos (≈ US$ 5–7 a preço de API)
busca por script	27%	1,6	—
ajuste dos truques de decodificação	48–52%	4,4	—
fine-tuning (LoRA)	29%	17,1	—

- **Fazer a correção é barato.** Uma execução de agente gasta mais ou menos a energia de um micro-ondas ligado por 15 minutos. A execução original do Astra usou 16× mais tokens lidos e 60× mais escritos, em 20 horas (ferramentas diferentes, então não é uma comparação controlada).
- **Usá-la economiza tokens em toda resposta,** mas só em relação ao raciocínio ligado. As respostas do modelo editado são 55% mais curtas no teste trancado; com o raciocínio desligado, o modelo sem edição é ainda mais curto.
- **Uma execução bem-sucedida não é a conta inteira.** Fizemos dez execuções neste defeito; as três com atlas foram piores.

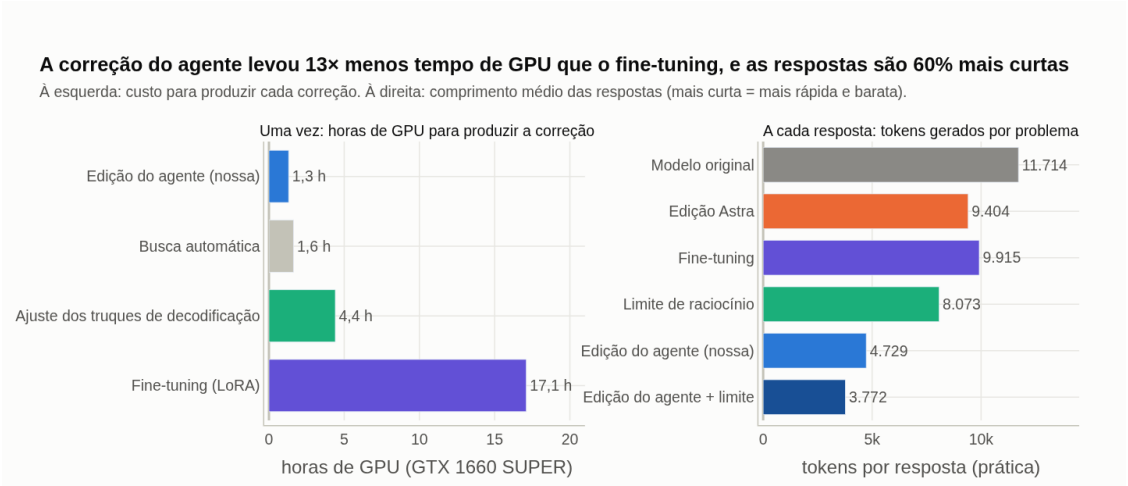


Figura 12: À esquerda, horas de GPU para produzir cada correção. À direita, o comprimento médio das respostas, que define o custo de cada resposta futura.

A cirurgia quebrou alguma coisa?

Nada que conseguíssemos detectar. Com o raciocínio desligado, nenhuma das edições muda o acerto em matemática (200 problemas) ou em chamada de ferramentas (300 pedidos) além de cerca de dois erros padrão; a versão de 2 edições que publicamos fica dentro de um. Esses painéis só detectam mudanças de uns 5 a 7 pontos.

Um custo: as respostas que terminam erram mais. No teste trancado, 74% das respostas terminadas da a1 estão certas, contra 92% do limite de raciocínio. Parte disso é seleção, já que as edições também terminam os problemas difíceis. Mas, nos problemas que os dois terminam, as edições com o empurrão de `</think>` mais forte continuam perdendo, e uma edição sem alavanca de `</think>` não perde. Elas fazem o modelo concluir antes da hora (análise post hoc, Apêndice A).

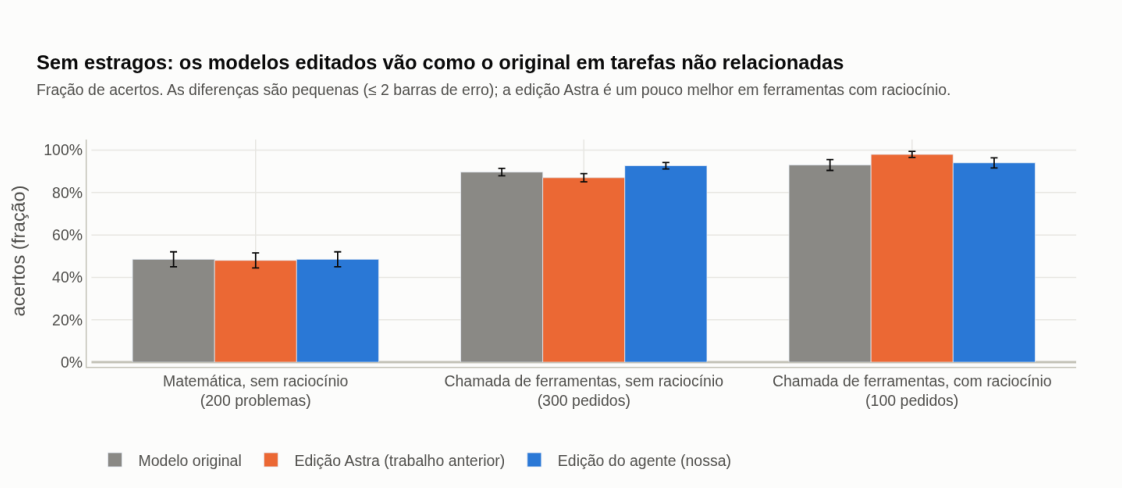


Figura 13: Acerto em tarefas não relacionadas para o modelo original, a edição publicada e a edição do agente.

Funciona num segundo defeito?

Não. Escolhemos seguir instruções (IFEval; Zhou et al., 2023: "sem vírgulas", "tudo em minúsculas"…), em que o modelo com raciocínio desligado segue todas as instruções em só cerca de 55% dos prompts, com falhas espalhadas por muitos tipos de instrução. Mesmo protocolo: um agente com 40 notas rápidas e 6 testes reais, contra um system prompt e fine-tuning.

Todo método pareceu melhor no próprio teste de prática (+5 a +12 pontos) e perdeu 3–7 pontos em prompts novos. O motivo é o ruído de amostragem: duas rodadas do modelo *sem mudança* discordam em 23% dos prompts, então um teste de 48 prompts tem margem de erro de 8–9 pontos, maior que qualquer ganho plausível. Nos loops, o efeito era de +46 pontos e 16 problemas bastavam para vê-lo. **Com um teste real tão ruidoso, o agente não consegue distinguir uma edição boa de uma que teve sorte.** Isso não contradiz o ganho no IFEval relatado para o Astra, medido com raciocínio ligado, onde os loops voltam a dominar. No caminho, o segundo agente do IFEval achou, só pelos pesos, um canal global de "esquecimento" que 99,7% das linhas de portão de decaimento de todas as 288 cabeças DeltaNet leem com o mesmo sinal (Gated DeltaNet; Yang et al., 2025). Editá-lo muda o modelo, mas não o seguimento de instruções.

Contexto e trabalhos relacionados

Programação no espaço de pesos. Takehi et al. (2026) mostraram que modelos de fronteira conseguem ler e escrever pesos de redes diretamente, incluindo a edição que conserta os loops e que reproduzimos.

Edição de modelos e steering. Edições de posto 1 mudam fatos guardados num modelo (ROME, Meng et al., 2022; MEMIT, Meng et al., 2023), apagar uma única direção pode remover a recusa (Arditi et al., 2024), e somar vetores em tempo de execução direciona o comportamento (Turner et al., 2023; Zou et al., 2023). Nossas edições são dessa família, mas um agente as escolhe raciocinando sobre o comportamento.

Pensar demais. Modelos de raciocínio costumam reconferir mais do que precisam (Chen et al., 2025). Suprimir "Wait" (Wang et al., 2025) e limitar o tempo de raciocínio (Muennighoff et al., 2025) são as nossas comparações.

Agentes e Goodhart. Agentes já rodam experimentos de interpretabilidade (Rott Shaham et al., 2024), e otimizadores fortes burlam objetivos substitutos (Gao et al., 2023; Skalse et al., 2022). Vemos o mesmo com indicadores de interpretabilidade.

Trabalhos concorrentes. Hu et al. (2026) rodam 16 trajetórias de edição do Astra e relatam grandes ganhos fora da amostra, sem comparações de decodificação ou fine-tuning. Reexecuções independentes acham que o raciocínio desligado com orçamento longo alcança boa parte do ganho do Astra sem editar (exp8-loop-controls, 2026; Zhang, 2026). Uma replicação caseira com um editor Claude achou o mesmo neurônio-freio, o L22n573, e viu que a edição dela e um orçamento maior de tokens se somam de forma quase independente no MATH-500 (tdotlab, 2026); outras replicações acharam edições diferentes (MatinKhajavi, 2026; ajaygrover, 2026), o que combina com o nosso achado de que várias edições tiram os loops. O IWIBench (von Csefalvay et al., 2026) avalia modelos editores. Nossa contribuição é a comparação controlada: edições de agente contra alternativas de decodificação, de raciocínio desligado e de fine-tuning em testes trancados, com o acerto por orçamento de tokens, contabilidade de custo, a evidência de que agentes burlam indicadores de interpretabilidade e uma fronteira medida de onde o método para de funcionar.

Limitações

- **Testes pequenos.** As comparações principais se apoiam em dois testes trancados de 200 problemas ($\pm$3–4 pontos cada); os números de prática e do MATH, em 48–148. Rodadas repetidas de um mesmo braço variam até 5,5 pontos.
- **Desligar o raciocínio é uma alternativa forte e barata.** As edições ficam 6,5–9 pontos acima dele nos 400 problemas trancados, com 2,4–3,6$\times$ os tokens, e empatam com ele na matemática mais difícil.
- **Um modelo, um defeito que funciona.** "Wait" é uma palavra fácil de mirar, e o defeito espalhado falhou. Não testamos um defeito concentrado sem palavra-chave, nem um segundo modelo.
- **O agente pode não ser necessário.** Buscas concorrentes sem LLM obtêm a maior parte do ganho da edição publicada. Não rodamos um script dirigido com o mesmo orçamento, nem comparamos modelos editores com várias execuções cada.
- **Braços tardios e regras por honra.** Alguns braços do primeiro teste rodaram depois da abertura (Apêndice A), e as regras dos agentes só foram verificadas depois, auditando os registros.

Dados, código e registros

Tudo está no repositório do projeto, https://github.com/luizmlo/agentic-weight-editing: o caderno de laboratório com previsões datadas (`docs/CADERNO.md`), os registros de cada execução de agente e de cada edição submetida (310), o notebook que desenha cada gráfico (`notebooks/results.ipynb`) e um estudo de custo (`docs/cost.md`). As transcrições completas dos agentes e as gerações brutas (~210 MB) serão publicadas à parte.

Uso de ferramentas de IA

Os agentes editores estudados aqui são o Claude Opus 5.5, e o Claude Fable 5.2 em duas execuções posteriores. O Claude (Opus 5.5 e Fable 5.2, pelo Claude Code) também foi usado como assistente de pesquisa: escreveu a maior parte do código, rodou os experimentos, manteve o caderno de laboratório, redigiu este texto e revisou as afirmações contra os dados brutos. O autor desenhou o estudo, supervisionou e conferiu o trabalho, e é responsável pelo conteúdo.

Apêndice A: Detalhes e estatística

Todos os valores de p são testes do sinal exatos sobre os pares discordantes, sem correção, salvo aviso.

Qual variante da a1 relatamos. O agente a1 ordenou duas variantes da mesma edição, com intensidades -9 e $-1{,}1$ (a primeira escolha dele) ou -6 e $-0{,}8$ (a segunda). Levamos a segunda, mais branda, a todas as comparações porque ela foi melhor no conjunto de prática (67% contra 65%); o agente a tinha posto em segundo, então a escolha foi nossa. No teste trancado as duas empatam (61,5% contra 62,0%), e a primeira escolha escreve menos da metade dos tokens.

Números do mecanismo. Com o texto fixo, os três neurônios da a1 baixam a log-probabilidade de "Wait" em 16,6 nats antes de cada "Wait", e a edição da última camada sobe a de `</think>` em 1,5 nat; o L18n881 quase não contribui. A a2 baixa "Wait" em 26 nats, tudo vindo do L16n998 ($-26{,}2$); a inversão do L22n573 sozinha *sobe* a palavra em 1,6. No modelo editado pela a2, a ativação do L22n573 antes de um "Wait" cai de $-3{,}73$ para $-0{,}04$. Nos testes reais com 48 problemas, a a2 de 2 edições resgata 6 e perde 4 contra a a2 inteira ($p = 0{,}75$); só o L16n998 resgata 3 e perde 13 contra ela ($p = 0{,}02$). No segundo teste trancado, a versão de 2 edições resgata 28 e perde 21 contra a a2 inteira ($p = 0{,}39$); na matemática mais difícil, 12 e 14.

Tabela completa de resultados.

método	conjunto de prática (48)	segundo conjunto de prática (32)	teste trancado 1 (200)	matemática mais difícil (148)
modelo original	15%	13%	15,5%	8%
modelo original, raciocínio desligado	—	—	58,5% †	44%
edição publicada (Astra)	46%	41%	53,5%	21%
proibir a palavra “Wait” (melhor de 3)	52%	47%	60,0% †	27% (primeiros 48)
limite de raciocínio, 4.096 tokens (melhor de 2)	48%	**63%**	61,0%	37%
limite, resposta com teto de 1.024 (melhor de 3)	50%	—	60,0% †	—
diferença de médias, sem agente (melhor de 6)	—	31%	—	—
busca por script, sem agente	27%	—	—	—
fine-tuning (LoRA)	29%	—	33,0% †	—
edição do agente (a1)	**67%**	56%	61,5%	**47%**
primeira escolha do agente a1	65%	—	62,0% †	—
edição do segundo agente (a2)	65%	—	67,5% †	49%
edição do agente + limite	**69%**	**63%**	**69,5%**	**53%**

† Rodado depois de o primeiro teste trancado ter sido aberto; nenhuma escolha dependeu destes braços.

Primeiro teste trancado, por orçamento de tokens (parcela resolvida se toda resposta for cortada nesse orçamento).

método	2.048	4.096	6.144	8.192	12.288	cortadas em 12k	tokens (média)
modelo original	3,5%	6,5%	10,5%	12,0%	15,5%	84,5%	11.142
edição publicada (Astra)	10,0%	19,0%	24,5%	39,0%	53,5%	42,5%	8.877
fine-tuning (LoRA) †	12,5%	20,5%	26,0%	30,0%	33,0%	65,5%	9.354
proibir a palavra “Wait” †	10,0%	26,5%	41,5%	50,5%	60,0%	35,5%	7.758
limite de raciocínio, 4.096	4,0%	6,5%	60,5%	60,5%	61,0%	34,0%	7.012
limite, resposta com teto de 1.024 †	2,5%	5,0%	60,0%	60,0%	60,0%	35,0% ‡	4.596
edição do agente (a1)	29,0%	45,0%	55,5%	60,5%	61,5%	17,0%	5.049
primeira escolha do agente a1 †	46,5%	60,0%	61,5%	62,0%	62,0%	5,5%	2.269
edição do segundo agente (a2) †	37,5%	53,0%	63,5%	66,5%	67,5%	15,5%	4.387
a1 + limite de raciocínio	25,0%	49,0%	67,5%	69,0%	69,5%	9,5%	4.081
a1 + limite, resposta com teto †	25,5%	54,5%	69,5%	69,5%	69,5%	15,0% ‡	3.172
modelo original, raciocínio desligado †	—	—	—	—	58,5%	5,0%	1.263

‡ Com teto para a resposta, esta coluna conta as respostas interrompidas por esse teto.

Neste teste, a a1 vence a edição publicada por 8,0 $\pm$ 3,4 pontos (32 resgates, 16 perdas; p = 0,03), e a a2 por 14,0 $\pm$ 3,6 (42/14). A a1 empata com o raciocínio desligado (+3,0 $\pm$ 3,5, p = 0,48); a a2 (+9,0 $\pm$ 3,5, p = 0,02) e a a1 + limite (+11,0 $\pm$ 3,5, p = 0,003) ficam acima dele. Com a correção de Holm sobre essas comparações, a a1 contra a edição publicada deixa de ser significativa; as outras duas continuam. A a1 + limite com teto fica acima da a1 sozinha (26/10, p = 0,01), do limite com teto sozinho (33/14, p = 0,008) e do raciocínio desligado (37/15, p = 0,003). Com o raciocínio desligado, as edições não ajudam nem atrapalham (a1 56,0%, edição publicada 57,5%).

Segundo teste trancado (GSM8K test[400:600]; todos os braços, comparações e previsões fixados antes da abertura).

método	segundo teste trancado	cortadas	tokens (média)	os dois testes (400)
modelo original, raciocínio desligado	58,0%	7,5%	1.365	58,3%
limite, resposta com teto de 1.024	59,5%	34,5% ‡	4.620	59,8%
edição do segundo agente (a2)	64,0%	15,5%	4.701	65,8%
a1 + limite, resposta com teto	64,5%	15,5% ‡	3.104	67,0%
a2 reduzida a 2 edições	67,5%	12,0%	4.217	—
edição do agente (a1)	68,0%	11,0%	4.459	64,8%

Sozinho, este teste não confirma nenhuma das cinco comparações primárias (todos os p ajustados por Holm = 0,83): todos os braços editados ficam à frente do raciocínio desligado na direção prevista (a2 +6,0 $\pm$ 4,0, a1 + limite com teto +6,5 $\pm$ 4,3), mas 200 problemas não confirmam diferenças de 5–7 pontos. Somando os 400 problemas, a a2 fica 7,5 $\pm$ 2,7 pontos acima do raciocínio desligado (74/44, p = 0,007) e a a1 + limite com teto 8,8 $\pm$ 2,8 (p = 0,002), as comparações secundárias pré-registradas; a a1 fica 6,5 $\pm$ 2,7 acima (p = 0,02, não pré-registrada). A a1 + limite com teto fica 7,2 $\pm$ 2,4 acima do limite com teto sozinho (p = 0,004, pré-registrada); a a1 e a a2, 5,0 e 6,0 acima dele (p = 0,047 e 0,02, não pré-registradas). A combinação fica só 2,2 $\pm$ 2,2 acima da a1 (p = 0,36).

Orçamento longo (20.480 tokens, primeiro teste trancado).

método	4.096	8.192	12.288	16.384	20.480	cortadas	tokens (média)
edição publicada (Astra)	21,5%	40,0%	51,0%	57,5%	65,0%	18,0%	11.213
edição do agente (a1)	49,5%	63,5%	67,0%	68,0%	68,0%	15,5%	6.608
edição do segundo agente (a2)	51,5%	61,0%	63,0%	63,5%	63,5%	15,0%	5.891

Com 20 mil, a edição publicada fica 11,5 pontos acima da rodada de 12 mil (31/8) e a menos de 3 pontos da a1 e da a2 (p $\geq$ 0,46). Cortadas em 12 mil, estas rodadas diferem das rodadas de 12 mil em $-$4,5 (a2), +5,5 (a1) e $-$2,5 pontos (Astra); o servidor rodou 8 pedidos em paralelo em vez de 16. Juntando as duas rodadas cortadas em 12 mil (400 respostas cada): a1 64%, a2 65%, edição publicada 52%.

Matemática mais difícil (MATH-500[0:148]). a1 contra a edição publicada: 44 resgates, 6 perdas; a2: 43/2; a1 contra a2: 15/18. a1 contra o limite de raciocínio +9,5 pontos (23/9); a1 + limite contra a a1 +6,1 (16/7, p = 0,09). O raciocínio desligado (teto de 8.192) resolve 43,9% com 3.842 tokens; a a1 (21/17, p = 0,63) e a a2 (21/14, p = 0,31) empatam com ele, e a a1 + limite (52,7%) fica acima (23/10, p = 0,035). O nosso corretor deixa passar algumas respostas equivalentes; corrigir os falsos negativos que achamos mantém a ordem (a1 49%, a1 + limite 55%, limite 38%, edição publicada 22%).

Terminadas e erradas (post hoc, Holm sobre 8 comparações). Nos 31 problemas que o modelo original termina, ele acerta 30; a primeira escolha da a1, 26, e a a1, 27. Nos problemas que os dois terminam, a primeira escolha contra o limite com teto: 4 resgates, 21 perdas (p = 0,001; Holm 0,008); contra a edição publicada, 5/16 (p = 0,03). A a1 contra o limite com teto, 3/13 (p = 0,02); a a2, 3/10 (p = 0,09); a f1, sem alavanca de `</think>`, 5/3.

Efeitos colaterais. Pareado com o modelo original, raciocínio desligado: chamada de ferramentas (300 pedidos) 90,3–91,0% para a a2 e a a2 reduzida contra 89,7%; matemática (200

problemas) 46,0–51,0% contra 48,5%; chamada de ferramentas com raciocínio 92–94% contra 93%. Com o raciocínio ligado, a edição publicada recusa melhor as chamadas de ferramenta irrelevantes (16 de 16 contra 11 de 16); as edições dos agentes não (10–12 de 16).

Braços do IFEval (mudança contra o original em prompts novos): agente b1 $-3{,}3 \pm 4{,}1$; a edição dos loops, a1, $-3{,}9 \pm 4{,}3$; system prompt $-6{,}6 \pm 4{,}2$; fine-tuning $-7{,}2 \pm 4{,}3$; agente b2 com testes de 128 prompts $-6{,}0 \pm 3{,}7$ (200 prompts). Nenhum braço difere do original por mais de dois erros padrão. Um segundo teste lacrado do IFEval continua fechado.

Braços tardios. A primeira escolha do agente a1, a a2, o raciocínio desligado, o limite com teto, a a1 + limite com teto, a proibição da palavra, o fine-tuning e a f1 rodaram no primeiro teste trancado depois da abertura. A proibição da palavra e o fine-tuning estavam no pré-registro original e só rodaram tarde; os outros foram pré-registrados depois da abertura, então a decisão de rodá-los pode refletir o que já tínhamos visto. O segundo teste trancado não tem braços tardios.

Apêndice B: Previsões que falharam

Anotamos previsões no caderno antes de cada resultado. Estas estavam erradas:

1. A edição com a melhor nota rápida (r1) superaria a edição publicada. Ela resolveu 4%.
2. Uma dose menor das direções do r1 recuperaria o ganho. Não recuperou.
3. O atlas ajudaria. Atrapalhou nos três pares.
4. A edição publicada superaria o limite de raciocínio na matemática mais difícil. O limite venceu (35% contra 21% em 48 problemas).
5. O primeiro agente do IFEval ganharia 5–9 pontos em prompts novos. Perdeu $3{,}3 \pm 4{,}1$.
6. O segundo agente do IFEval, com testes maiores, ganharia 5–15 pontos. Perdeu $6{,}0 \pm 3{,}7$.
7. A a2 faria 52–64% no teste trancado. Fez 67,5%.
8. Pelo menos uma execução do Fable editaria o L22n573 ou outro neurônio-freio de "Wait" nas camadas 17–23. Nenhuma mirou neurônios isolados; as duas editaram camadas inteiras.
9. Proibir a palavra faria 40–52% no teste trancado. Fez 60,0%.
10. A a1 + limite com teto venceria o raciocínio desligado com $p < 0{,}05$ no segundo teste trancado (70%). Ficou 6,5 pontos à frente, $p = 0{,}17$.
11. A a1 faria 55–67% no segundo teste trancado. Fez 68,0%.
12. A a2 reduzida usaria pelo menos tantos tokens quanto a a2 ali (60%). Usou 10% menos.
13. Com o raciocínio desligado, a matemática mais difícil usaria menos de um terço dos tokens da a1 (85%). Usou 46%.
14. Pelo menos uma das duas réplicas do Opus (a3, a4) chegaria a 60% no conjunto de prática (75%). As duas resolveram 58%.

Glossário

- **Modelo de linguagem, pesos:** o modelo é uma grande coleção de números (os *pesos*) organizados em matrizes. O Qwen3.5-0.8B tem cerca de 800 milhões deles. Treinar muda os pesos aos poucos, com gradientes. Aqui, um agente muda poucos pesos de forma direta e deliberada.
- **Token:** o pedaço de texto que o modelo lê e escreve de cada vez, cerca de ¾ de uma palavra.
- **Raciocínio (“thinking”), loop:** antes de responder, o modelo escreve um rascunho entre `<think>` e `</think>`. Ele está em *loop* quando repete o mesmo trecho (“Espera, deixa eu conferir…”) até esgotar o espaço.

- **Edição de posto 1 (rank-1):** a menor mudança estruturada numa matriz. Ela escolhe uma direção e um grupo de *neurônios* (colunas da matriz) e muda só o quanto esses neurônios escrevem naquela direção. Cada correção usou no máximo 4 dessas edições.
- **Fine-tuning, LoRA:** o caminho usual, que é treinar o modelo com exemplos. LoRA é a versão barata, que treina só pequenas matrizes acrescentadas ao modelo.
- **Steering (direcionamento):** somar um vetor às ativações internas para empurrar o comportamento. A edição Astra é desse tipo, só que gravada nos pesos.
- **Decodificação:** como o texto é sorteado a partir das probabilidades do modelo. *Proibir a palavra* (logit bias) e *limite de raciocínio* (budget forcing) mudam a decodificação, não os pesos.
- **Agente:** um modelo de IA (aqui, Claude) que trabalha sozinho com ferramentas: lê os pesos, propõe edições, pede avaliações e decide o próximo passo.
- **Nota rápida (sonda) e teste real:**
  - a nota rápida mede, em segundos, uma probabilidade interna (por exemplo, o quanto o modelo "quer" repetir);
  - o teste real faz o modelo editado de fato resolver problemas e confere as respostas. Lição central: notas rápidas podem ser burladas, e só o teste real é confiável.
- **Conjunto de prática, teste trancado, pré-registro:** os agentes só viam 16 problemas de prática; confirmamos as escolhas deles em outros 48. O *teste trancado* fica lacrado até que cada método tenha escolhido sua versão final. As previsões foram escritas no caderno *antes* de abrir os resultados (pré-registro).
- **Erro-padrão ($\pm$ EP), comparação pareada:** "+8,0 $\pm$ 3,4" quer dizer um ganho de 8 pontos com incerteza de cerca de 3,4. Uma diferença maior que ~2 EP raramente é sorte. Na comparação pareada, os dois métodos fazem os *mesmos* problemas: *resgates* são os que só o primeiro acerta, e *perdas*, os que só o segundo acerta.
- **GSM8K, MATH-500, IFEval:** conjuntos públicos de avaliação.
  - GSM8K: matemática do ensino fundamental.
  - MATH-500: matemática de competição.
  - IFEval: seguir instruções verificáveis ("sem vírgulas", "tudo em minúsculas"…).
- **Lei de Goodhart:** quando uma medida vira meta, ela deixa de ser uma boa medida. É o que aconteceu com as notas rápidas.
- **Atlas:** um mapa extra da atividade interna do modelo (quais neurônios disparam em loops), dado a metade dos agentes.
- **DeltaNet:** o tipo de camada ("atenção linear") que forma 18 das 24 camadas do Qwen3.5. Ela guarda uma memória resumida do texto e tem *portões* que decidem o quanto esquecer.

Referências

- ajaygrover (2026). weight-space-programming-stacking. GitHub repository, https://github.com/ajaygrover/weight-space-programming-stacking. Accessed Nov. 10, 2026.
- Arditi, A., Obeso, O., Syed, A., Paleka, D., Panickssery, N., Gurnee, W., Nanda, N. (2024). Refusal in Language Models Is Mediated by a Single Direction. NeurIPS 2024. arXiv:2406.11717.
- Chen, X., Xu, J., Liang, T., He, Z., et al. (2025). Do NOT Think That Much for 2+3=? On the Overthinking of Long Reasoning Models. ICML 2025, PMLR 267. arXiv:2412.21187.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.
- exp8-loop-controls (2026). Pre-registered controls for Experiment 8 of Sparks of Weight-Space Programming. GitHub repository, https://github.com/1099271/exp8-loop-controls. Accessed Nov. 10, 2026.
- Gao, L., Schulman, J., Hilton, J. (2023). Scaling Laws for Reward Model Overoptimization. ICML 2023. arXiv:2210.10760.
- Gerganov, G., and contributors (2023–). llama.cpp. Software. https://github.com/ggml-org/llama.cpp.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. NeurIPS 2021 Datasets and Benchmarks Track. arXiv:2103.03874.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. arXiv:2106.09685.
- Hu, J., Tang, C., Li, W., Zhu, H., et al. (2026). Exploring Weight-Space Programming: Capabilities, Generalization, and Limitations. arXiv:2611.09382.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., et al. (2024). Let's Verify Step by Step. ICLR 2024. arXiv:2305.20050.
- MatinKhajavi (2026). Weight-Space-Programming: a replication with Claude Opus 5.5. GitHub repository, https://github.com/MatinKhajavi/Weight-Space-Programming. Accessed Nov. 10, 2026.
- Meng, K., Bau, D., Andonian, A., Belinkov, Y. (2022). Locating and Editing Factual Associations in GPT. NeurIPS 2022. arXiv:2202.05262.
- Meng, K., Sen Sharma, A., Andonian, A., Belinkov, Y., Bau, D. (2023). Mass-Editing Memory in a Transformer. ICLR

  2023. arXiv:2210.07229.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., et al. (2025). s1: Simple test-time scaling. EMNLP 2025. arXiv:2501.19393.
- nathanrs (2026). weight-space-lp. GitHub repository, https://github.com/nathanrs/weight-space-lp. Accessed Nov. 10, 2026.
- Qwen Team (2026). Qwen3.5: Towards Native Multimodal Agents. Blog post, https://qwen.ai/blog?id=qwen3.5. Model: https://huggingface.co/Qwen/Qwen3.5-0.8B.
- Rott Shaham, T., Schwettmann, S., Wang, F., Rajaram, A., Hernandez, E., Andreas, J., Torralba, A. (2024). A Multimodal Automated Interpretability Agent. ICML 2024. arXiv:2404.14394.
- Skalse, J., Howe, N. H. R., Krasheninnikov, D., Krueger, D. (2022). Defining and Characterizing Reward Gaming. NeurIPS 2022. arXiv:2209.13085.
- Takehi, R., Teoh, J., Zhang, A., Khattab, O. (2026). Sparks of Weight-Space Programming: LLMs Can Natively Read & Write Neural Network Weights. Blog post, MIT CSAIL, Nov. 2, 2026. https://rikiyat.github.io/blog/weight-space-programming/. Logs: https://github.com/RikiyaT/lm_understand-logs.
- tdotlab (2026). weight-space-programming: a home replication with a Claude editor. GitHub repository, https://github.com/tdotlab/weight-space-programming. Accessed Nov. 10, 2026.
- Turner, A. M., Thiergart, L., Leech, G., Udell, D., Vazquez, J. J., Mini, U., MacDiarmid, M. (2023). Steering Language Models With Activation Engineering. arXiv:2308.10248.
- von Csefalvay, C., Doma, P., Unni Krishnan, A., Földi, T. (2026). IWIBench: Evaluating Gradient-Free Model Driven Intra-weight Injection. arXiv:2611.08647.
- Wang, C., Feng, Y., Chen, D., Chu, Z., Krishna, R., Zhou, T. (2025). Wait, We Don't Need to "Wait"! Removing Thinking Tokens Improves Reasoning Efficiency. Findings of EMNLP 2025. arXiv:2506.08343.

- WKJBryan (2026). wsp-lab. GitHub repository, https://github.com/WKJBryan/wsp-lab. Accessed Nov. 10, 2026.
- Yang, S., Kautz, J., Hatamizadeh, A. (2025). Gated Delta Networks: Improving Mamba2 with Delta Rule. ICLR 2025. arXiv:2412.06464.
- Zhang, Y. (2026). Rethinking Weight-Space Programming. Blog post and repository, https://github.com/ZhangYiqun018/rethinking-weight-space-programming. Accessed Nov. 10, 2026.
- Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., Hou, L. (2023). Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911.
- Zou, A., Phan, L., Chen, S., Campbell, J., et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.